

nano-gpt

Sıfırdan Karakter Seviyeli GPT (Transformer)

Andrej Karpathy'nin "Let's build GPT" dersinin PyTorch ile birebir, bol Türkçe yorumlu uygulaması. Self-attention, multi-head, residual bloklar ve decoder-only bir dil modelinin tüm parçaları.

PyTorch

Transformer

Self-Attention

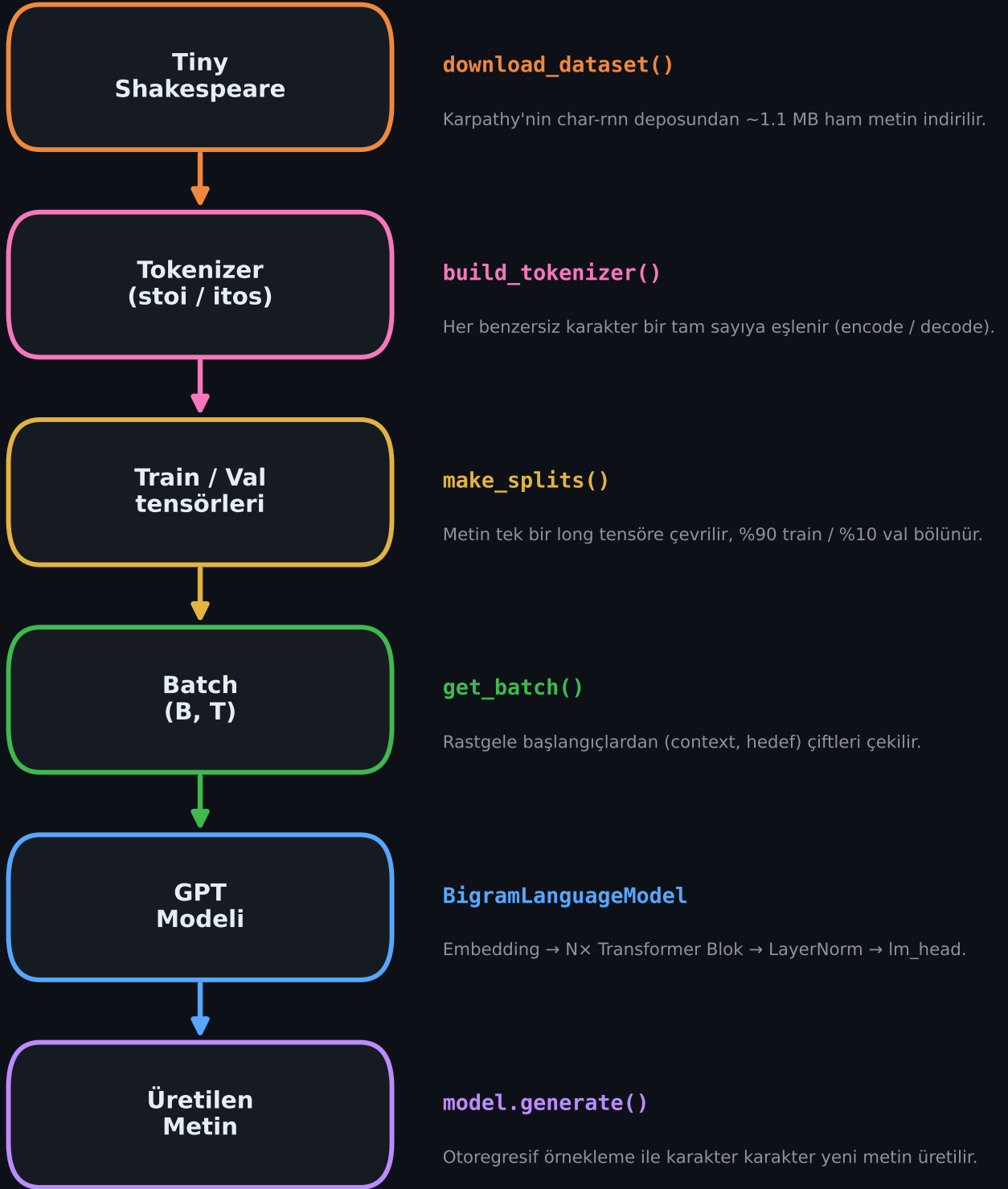
Char-level

Weights & Biases

Bir Bakışta Mimari

Veri kümesi	Tiny Shakespeare (~1.1 MB metin)
Tokenizer	Karakter seviyeli · vocab $\approx$ 65
Model	Decoder-only Transformer · 6 blok · 8 head
Gömme boyutu	n_embed = 256 · block_size = 256
Eğitim	AdamW + Cosine LR · Early Stopping · W&B

Ham metinden üretilen yeni metne kadar tüm boru hattı. Her kutu, notebook'taki bir fonksiyon veya modüle karşılık gelir.



Tokenizer ve Batch Üretimi

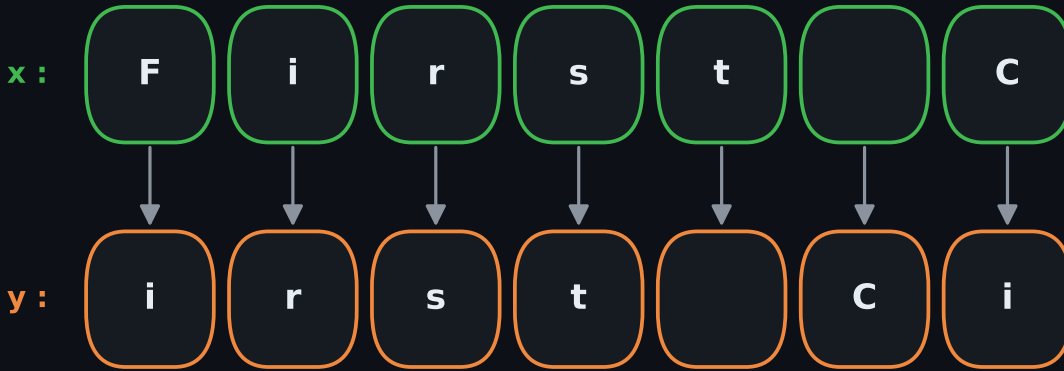
1 · Karakter Seviyeli Tokenizer

BPE yerine her karakter tek bir id. Vocab küçük (~65), embedding tablosu küçük, hata ayıklaması kolay.



2 · get_batch → (context, hedef) çiftleri

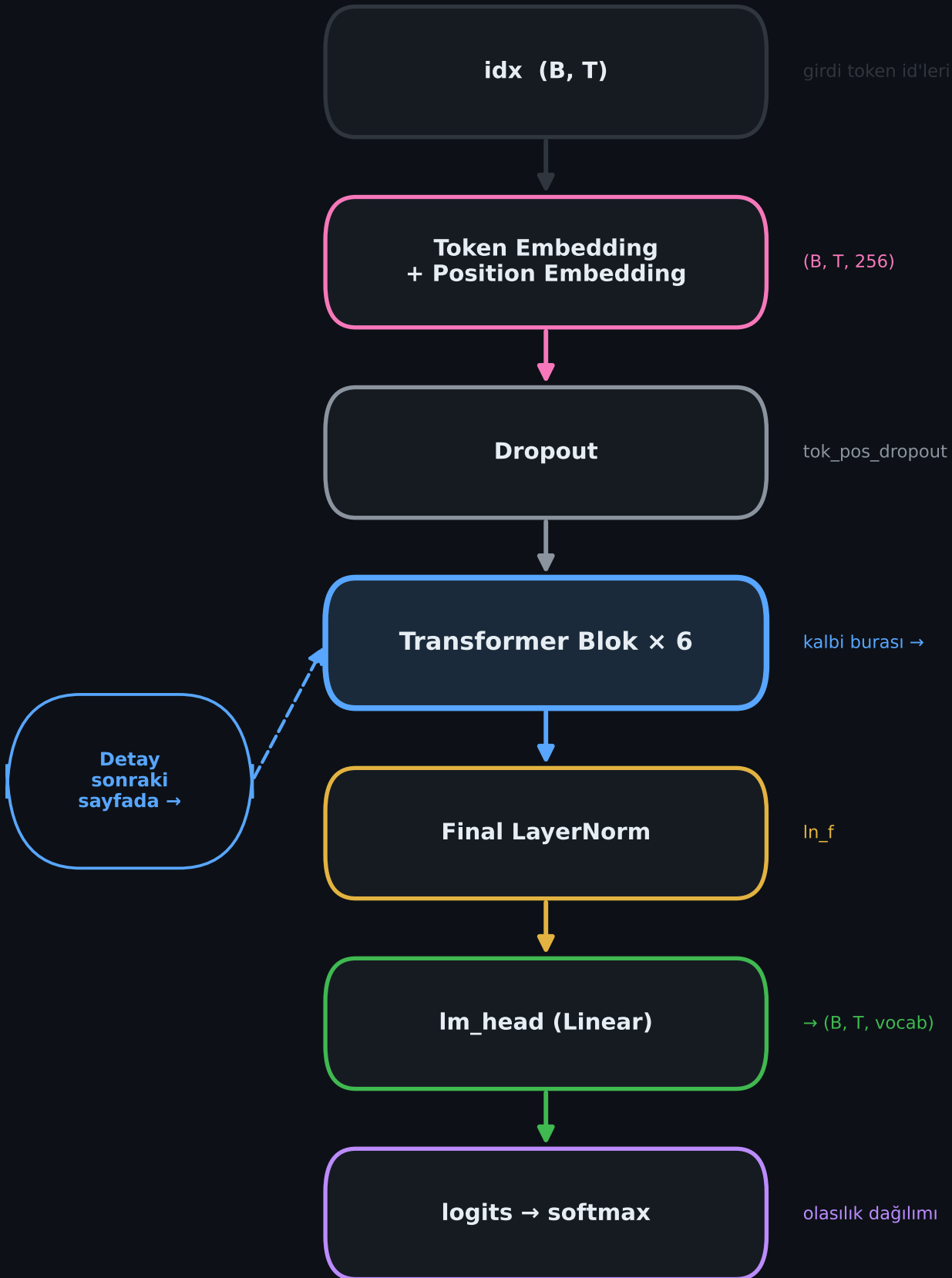
block_size uzunluğundaki bir x dilimi, 1 sağa kaymış y ile eşleşir. Tek dilim, T adet bağımsız eğitim örneği taşır — Transformer'ın paralel öğrenmesinin kaynağı budur.



tahmin yönü: $x[:, t]$ verildiğinde model $y[:, t]$ 'yi tahmin eder

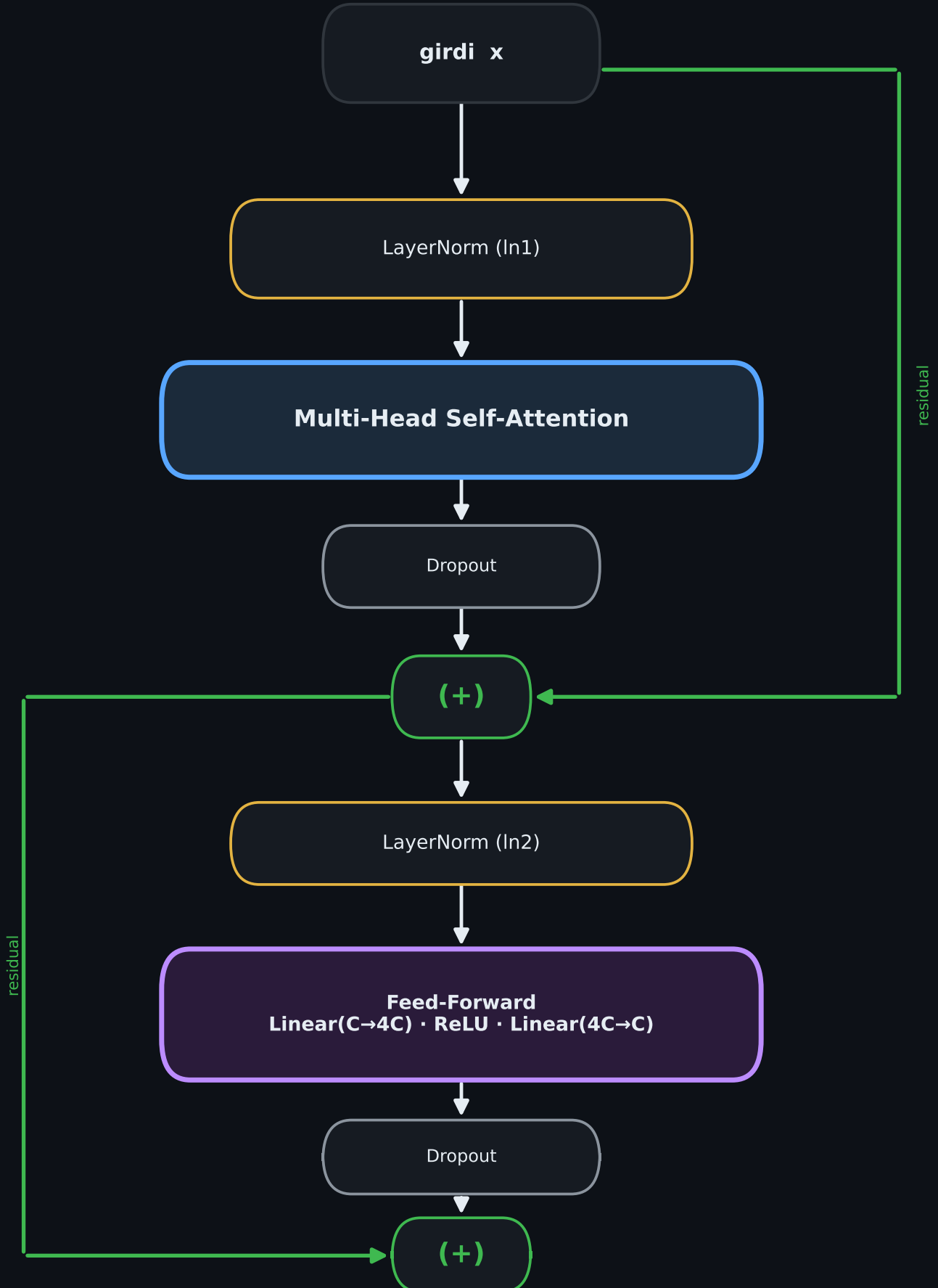
Tensör Şekilleri

x, y	(B, T)	B = batch_size, T = block_size
token_embedding	(B, T, C)	C = n_embed
logits	(B, T, vocab_size)	her pozisyon için skor
loss	skaler	cross-entropy



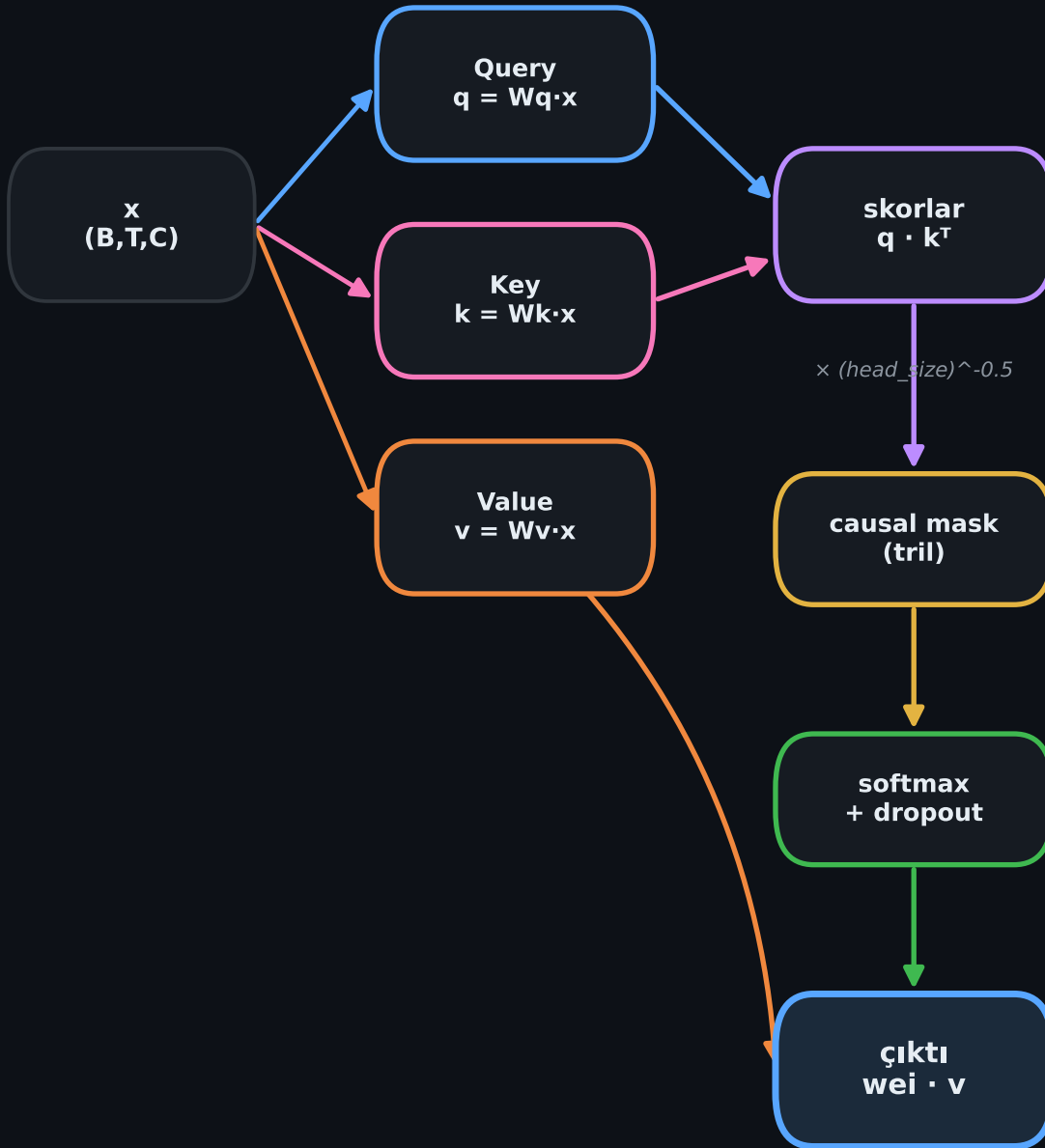
Transformer Bloğu — Haberleşme + Hesaplama

Her blokta iki alt-katman vardır. Pre-LayerNorm uygulanır ve her alt-katmanın çıktısı residual (artık) bağlantıyla girdiye eklenir: $x = x + \text{Dropout}(\text{SubLayer}(\text{LN}(x)))$.



Tek Bir Self-Attention Head

Her head, girdiyi üç projeksiyona ayırır: Query, Key, Value. Skorlar ölçeklenir, causal mask ile gelecek maskelenir, softmax ile ağırlığa çevrilir ve Value'lar toplanır.



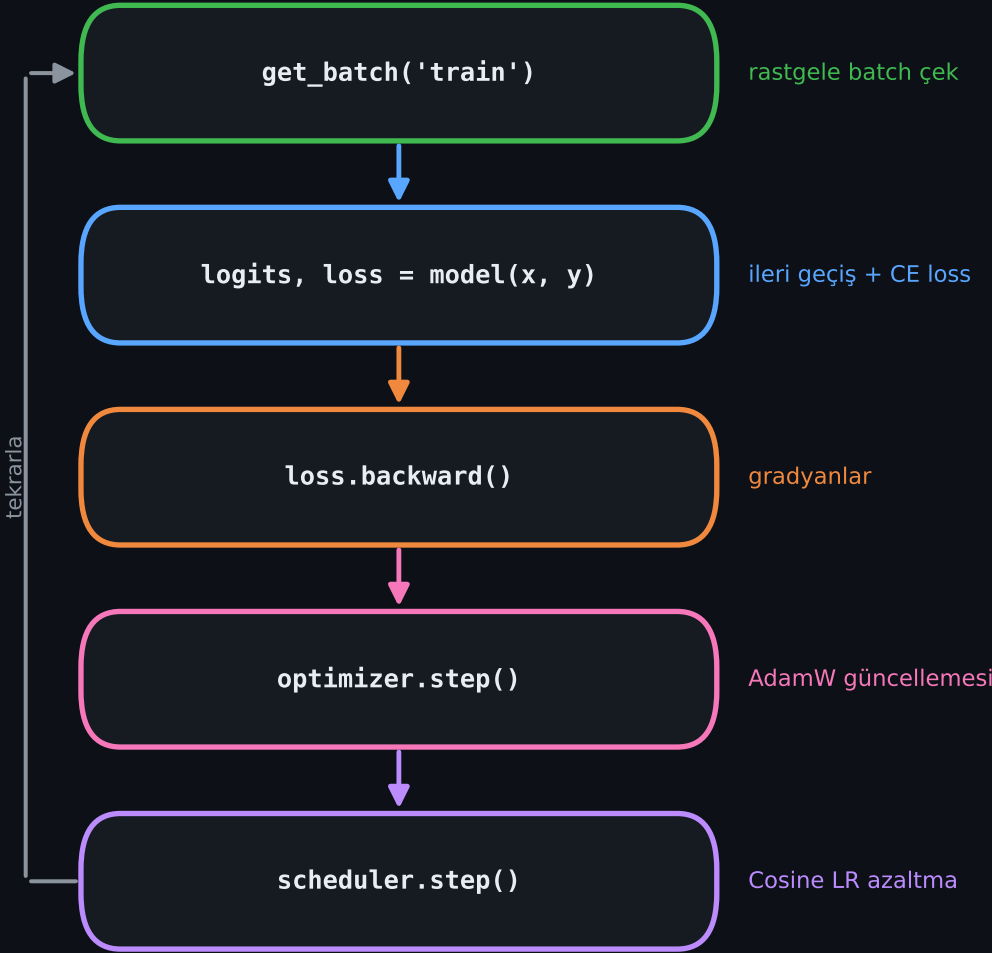
Causal mask: bir token yalnızca kendisini ve geçmişi görebilir (gelecek = $-\infty$)

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	1	1	1	1

satır = sorgulayan pozisyon
sütun = bakılan pozisyon

yeşil (1) = izinli
koyu (0) = maskeli

İterasyon Döngüsü



Düzenleme & İzleme

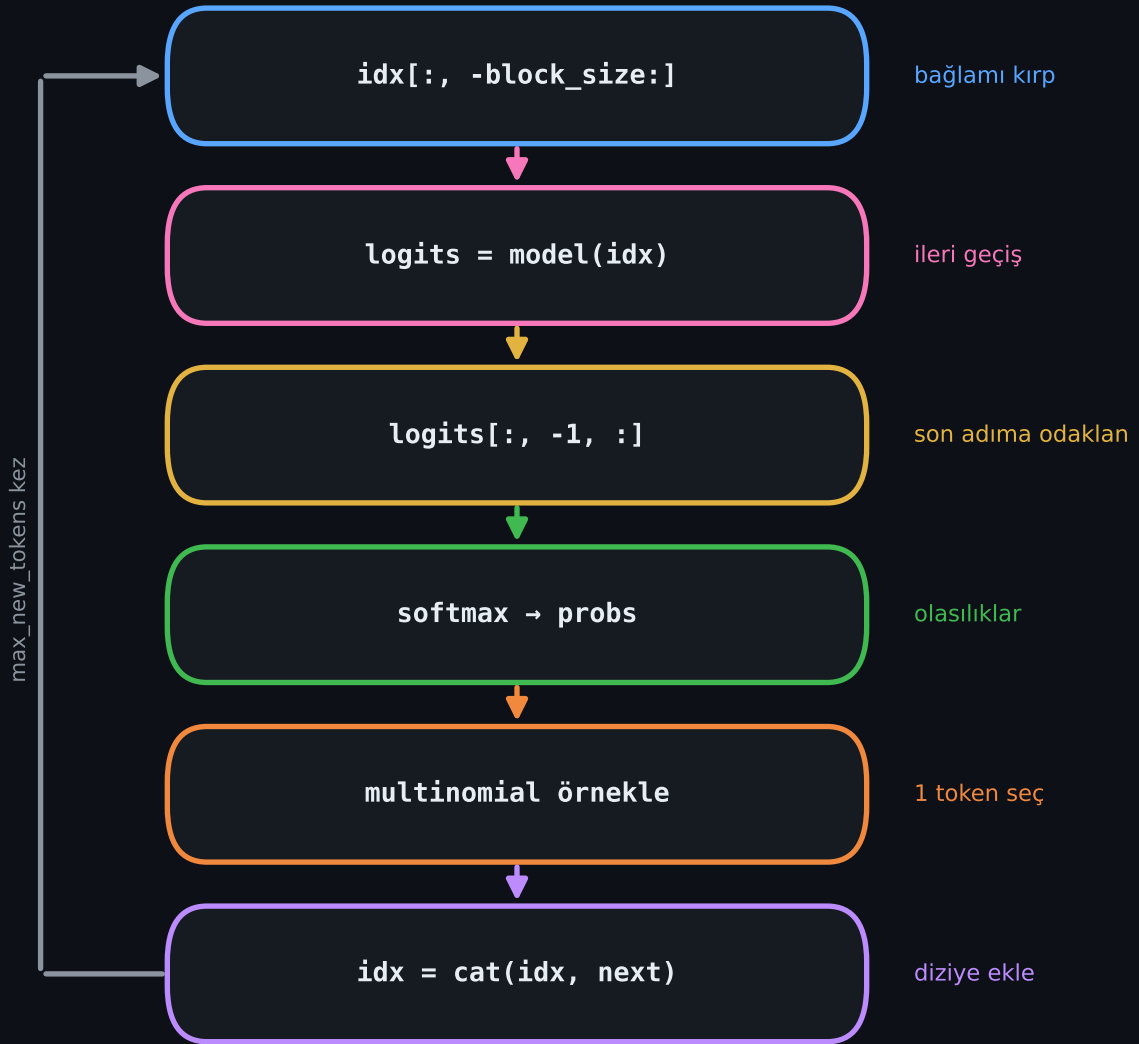
- **estimate_loss**: her 500 adımda train/val loss tahmini (eval modu)
- **Early Stopping**: val loss 5 değerlendirme boyunca düşmezse durur
- **Cosine LR**: lr 5e-4 → 1e-5, yumuşak iniş
- **Weights & Biases**: tüm metrikler canlı loglanır

HERO RUN v4 — Konfigürasyon

block_size	256	batch_size	64	n_embed	256
n_head	8	n_layer	6	dropout	0.2
lr	5e-4	epochs	10000	weight_decay	1e-3

Otoregresif Metin Üretimi

`model.generate()` her adımda son `block_size` token'ı bağlam olarak alır, son pozisyonun olasılık dağılımından bir token örnekler ve diziye ekler. Bu döngü kendi kendini besler.



Neden Önemli?

Bu küçük model, modern büyük dil modellerinin (GPT) tam olarak aynı çekirdek mekanizmasını içerir: token + pozisyon gömme, causal self-attention, multi-head, residual bloklar ve otoregresif üretim. Ölçek dışında temel mimari aynıdır — bu yüzden "sıfırdan GPT" öğrenmek için ideal bir başlangıçtır.